

AeroMed DevOps Platform

Architecture & Design Document

Case Study 75 — Critical Care Air Ambulance Operations Platform

Cloud-Native · Highly Available · Disaster-Recovery Ready

Microservices · Kubernetes (EKS) · Terraform · Jenkins · Prometheus / Grafana

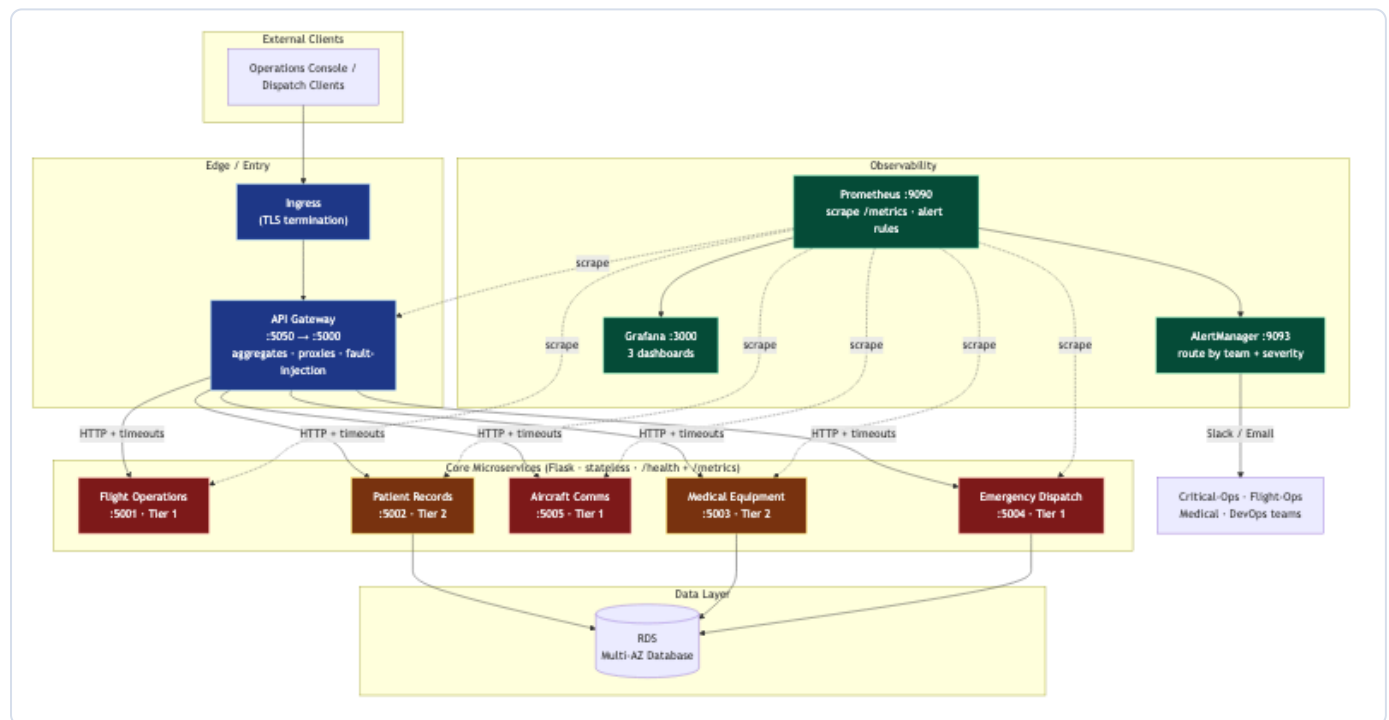
Executive Overview

AeroMed is a production-grade DevOps platform for a global air ambulance network, where infrastructure failure is a patient-safety event. The platform is engineered around three principles: **high availability** (multi-replica, self-healing, autoscaling), **observability** (every service exposes health and Prometheus metrics), and **provable disaster recovery** (tiered RTO/RPO with six tested runbooks). Six fault-isolated microservices run on Kubernetes behind a single API Gateway, delivered through Jenkins CI/CD onto Terraform-provisioned AWS infrastructure.

Deliverables Status

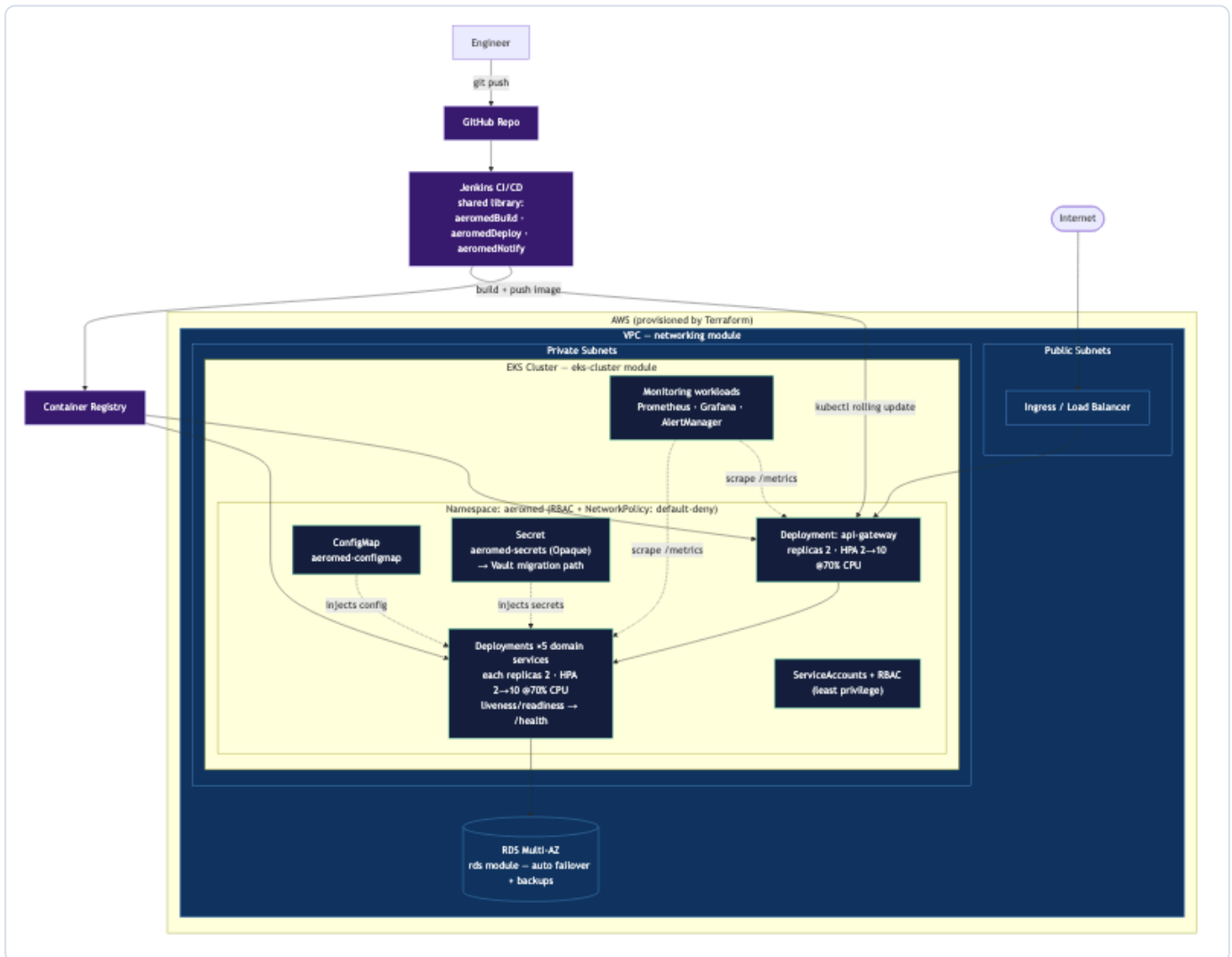
Deliverable	Status	Evidence / Notes
Working Application	Delivered	6 Flask microservices
Dockerfiles & Images	Delivered	6 Dockerfiles, images running
Jenkins CI/CD	Delivered	Master + 6 pipelines + shared library
Terraform IaC	Delivered	eks-cluster, networking, rds, monitoring
Kubernetes Manifests	Delivered	Deploy, svc, configmap, secret, hpa, ingress, rbac, netpol
Prometheus + Grafana	Delivered	3 dashboards, alert + recording rules
Disaster Recovery Plan	Delivered	6 runbooks + RTO/RPO + scripts
Architecture & Deployment Diagrams	Delivered	This document
Project Documentation	Delivered	docs/ + README
ELK Stack (logging)	Planned	Structured stdout today; ELK/Loki next increment
Vault (secrets)	Planned	K8s Secrets today; documented Vault Agent Injector path
Demonstration Screenshots	Pending	Capture via demo-full-recovery.sh + Grafana

1. System Architecture (Logical)



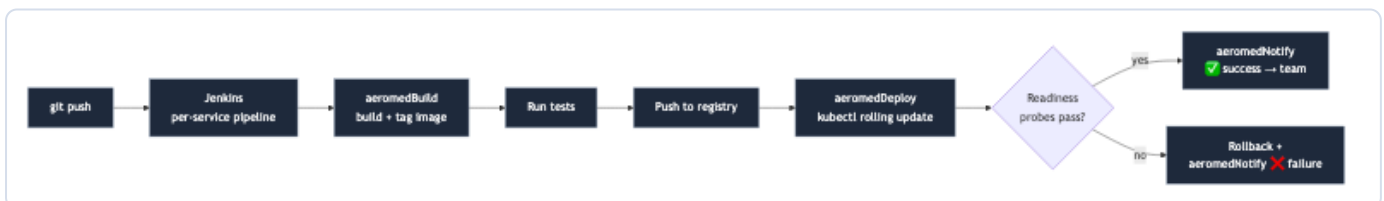
Six independently deployable Flask microservices behind a single internet-facing API Gateway. All gateway→service calls use explicit HTTP timeouts for graceful degradation. Services are colour-coded by disaster-recovery tier (Tier 1 = red, 5-min RTO; Tier 2 = amber, 15-min RTO). Prometheus scrapes every /metrics endpoint out-of-band, so the monitoring stack is never in the request path.

2. Deployment Architecture (Kubernetes on AWS EKS)



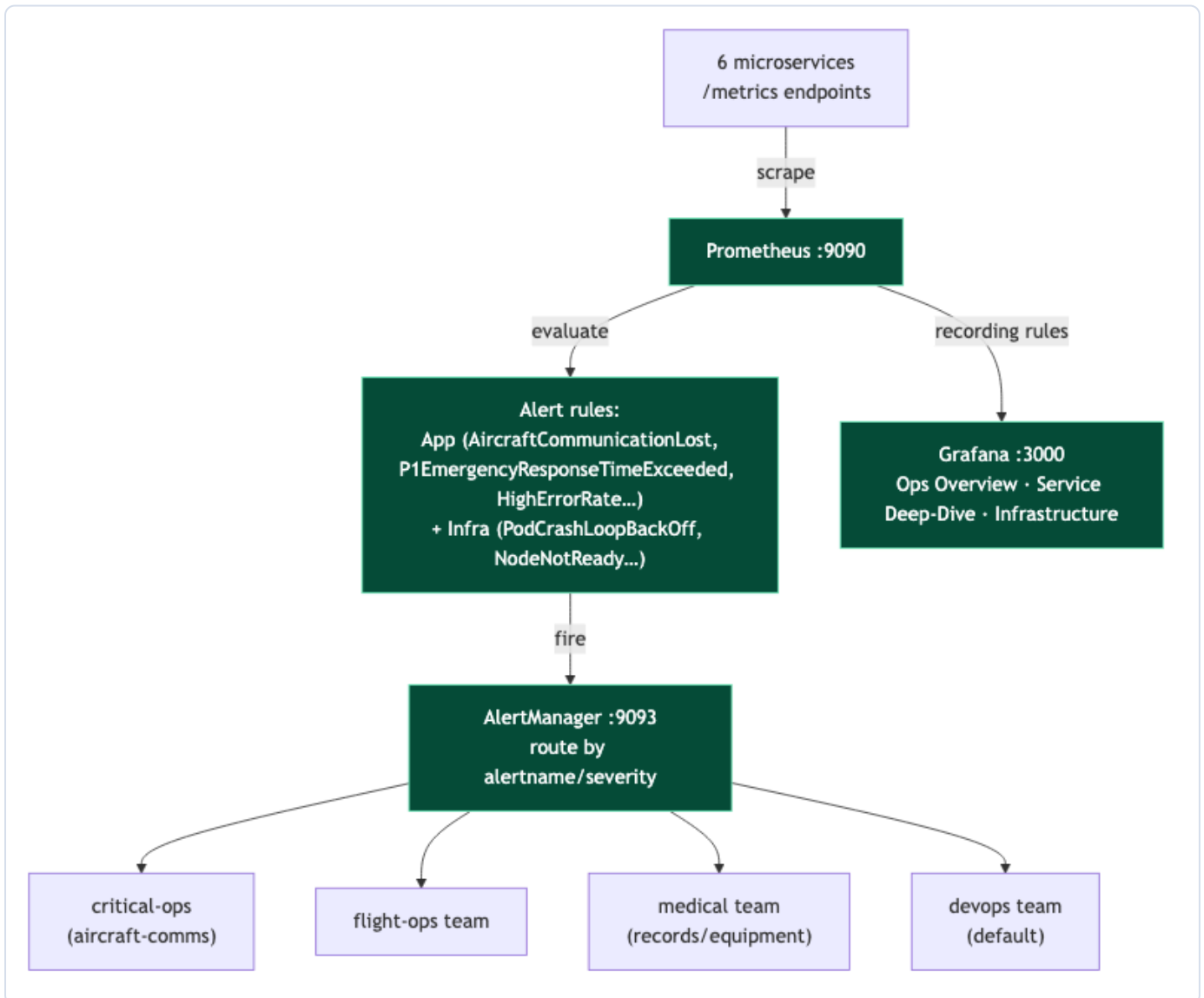
Runtime topology provisioned by Terraform: a VPC with public/private subnet separation, an EKS cluster, and an aeromed namespace governed by RBAC and a default-deny NetworkPolicy. Every service runs a minimum of 2 replicas with an HPA scaling 2→10 at 70% CPU, liveness/readiness probes on /health, ConfigMap and Secret injection, and a Multi-AZ RDS database. Jenkins builds images and performs zero-downtime rolling updates.

3. CI/CD Pipeline Flow



Each service pipeline reuses three shared-library steps — aeromedBuild, aeromedDeploy, aeromedNotify — defined once and inherited everywhere. Failed readiness probes trigger automatic rollback and a failure notification.

4. Monitoring & Alert Routing



Prometheus evaluates application alerts (AircraftCommunicationLost, P1EmergencyResponseTimeExceeded, HighErrorRate) and infrastructure alerts (PodCrashLoopBackOff, NodeNotReady). AlertManager routes by alertname and severity so the correct team — critical-ops, flight-ops, medical, or devops — is paged for each failure.

5. Disaster Recovery Map

Tier 1 – RTO 5 min

RB-003
Aircraft Comms Loss

AircraftCommunicationLost
alert
→ K8s self-heal →
operator verify

RB-001
Service Pod Failure

RB-002
Pod CrashLoopBackOff

Tier 2 – RTO 15 min

RB-004
Database Failure

RDS Multi-AZ auto-failover
→ restore.sh / verify
integrity

Tier 3 – RTO 30 min

RB-005
Full Cluster Failure

Terraform re-apply
→ rebuild region from code

RB-006
Traffic Surge / Mass
Casualty

HPA scales 2→10
→ absorb surge
automatically

Six numbered runbooks (RB-001...RB-006) mapped to recovery tiers, RTO targets, and recovery mechanisms — from aircraft-comms self-healing, to RDS Multi-AZ failover, to full-region rebuild from Terraform, to HPA-driven surge absorption.